Performance Trade-offs in Static and Dynamic Thread Pool Architectures

Harshil Patel
University of California, Davis
Department of Computer Science
USA

Anugya Sharma
University of California, Davis
Department of Computer Science
USA

Kaichi Xie
University of California, Davis
Department of Computer Science
USA

Alfredo Ortiz
University of California, Davis
Department of Computer Science
USA

1 Introduction & Motivation

The advancements in multicore architecture and multiprocessor computers are increasing and ongoing [1]. Modern concurrent systems have to choose between static thread pools (pre-created threads) and dynamic thread pools (threads created on demand), but they lack the empirical data to make informed decisions. Static thread pools waste memory during low load and idle periods which results in a constant overhead regardless of the workload. Dynamic thread pools avoid this waste by creating threads only when needed, but they incur overhead for thread creation and destruction, which can lead to performance degradation under high load. In this project, we want to evaluate the performance trade-offs between static and dynamic thread pools under varying workloads and system configurations and answer the question: At what point does the overhead of dynamic thread creation outweigh its memory savings compared to static thread pools?

This tradeoff can directly impact the performance and costs of cloud-based systems [2]. A web service handling variable traffic has to decide between a static pool with constant resource usage or a dynamic pool that adapts to demand but may suffer from latency spikes during high traffic. As cloud providers charge based on resource usage, understanding this tradeoff can lead to significant cost savings while maintaining performance. Apart from costs, this question also affects other aspects of system design such as database connection pools, task scheduling in distributed systems, etc which all use thread pools to manage concurrency. Yet, there is a lack of comprehensive studies that quantify the performance implications of static vs dynamic thread pools across different workloads and system configurations. By systematically evaluating these trade-offs, we can provide practical guidelines for system architects to choose the right thread pool strategy based on their specific workload and performance requirements.

2 Background & Related Work

Existing research mainly focuses on optimizing thread pool sizes rather than comparing different thread pool architectures. Current heuristics that are used to determine the number of threads are flawed [6]. In our preliminary research, we find that there have been few researches on dynamic model for thread size. One of the first such model was the Watermark model which adjusts the number of threads according to the current number of incoming requests [4]. The issue with this model is that while it solves the problem of "efficient usage of resources", it doesn't efficiently obtain the required number of threads in advance [3]. Furthermore, there are also prediction based models to determine thread pool size, but in such models, only factors such as the request rate or number of worker threads is considered in the management [5]. One of the research that has proposed dynamic thread pool model(prediction based), themselves point out that the model lacks testing for a longer period of time and on general server environment which provides various services simultaneously [6]. Lee et al. [5] adds to the prediction model by proposing what they call the "Trendy exponential moving average model" and again the issue with this model seems to be that it doesn't have the desired accuracy in its prediction.

A comprehensive study comparing the tradeoffs between static and dynamic thread pools under varying workloads and system configurations is missing. This tradeoff can directly impact the performance and costs of cloud-based systems [2]. Freire et al. [2] compare global thread pools (single pool and queue for all tasks) against local thread pools (each task has its own dedicated pool and queue). Their study simulates these configurations under high workloads to demonstrate that optimized local pools result in a lower average makespan when compared to global pools. While Freire et al. [2] provide insights into thread pool configurations, they focus on optimal distribution of a fixed number of threads, while our work aims to compare static vs dynamic thread pool architectures across varying workloads and system settings. By systematically evaluating these trade-offs, we can provide practical guidelines for system architects to choose the right thread pool strategy based on their specific workload and performance requirements.

3 Implementation

We tested the dynamic thread pool and the static thread pool on cloud lab. We believe that testing the thread pool on cloud lab would give the most accurate results as cloudlab machines are bare-metal and will not have any background processes that would normally be running on a normal computer.

3.1 Hardware

The system we used on cloud had two Intel Xeon Silver 4114 CPUs and each one has 10 cores. Although, they also have hyper threading technology so each core logically acts has another core. So we potentially had 40 cores we could use for this project research. Each

core can run at a base frequency of 2.20 GHz, and can reach a max turbo frequency of 3.00 GHz. The system also had 192GB of DDR4 RAM, which was plenty for the project. The operating system that was provided by cloud lab was Ubuntu 24.04.

3.2 Thread Pool Implementation

When we implemented the thread pool we wanted to make sure that the comparison between the two pools were completely fair. One thread pool should not have an upright advantage over the other since we want to get an accurate result based on the methodology of the thread creation. To achieve this we created a task queue which was used on both static thread pool and dynamic thread pool. Task queue ensures that both thread pools are thread safe, and ensures that jobs are being queued are exactly the same, so that there is fairness between the two programs. The static thread pool will be fixed at ten threads when when running the test. The dynamic thread pool will start with two threads, and will be able to expand to the capacity of what the system can provide. This will allow us to see how will dynamic thread pool can perform under a high load scenarios. The dynamic thread pool will shrink its pool if 500ms has elapsed and there is no job for a thread to do, this will allow use to test how dynamic thread pool can save system resources.

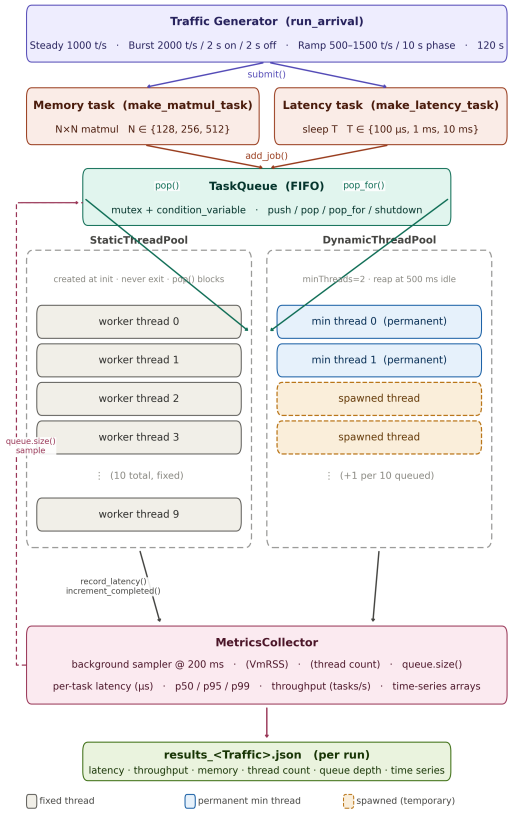


Figure 1: Thread Pool Architecture

3.3 Evaluation Setup

To evaluate our thread pool implementations, we benchmarked them under three traffic patterns, each defined by a task arrival process that reflects different traffic scenarios:

- Steady load: a constant arrival rate of 1000 tasks/second for the full duration of the run.
- Burst Load: a burst of 2000 tasks/second followed by a 2-second idle period, repeated in cycles.
- Ramping Load: alternating phases on 1500 tasks/second and 500 tasks/second in repeated cycles.

To isolate how task characteristics interact with thread-pool policy, we will apply two controlled micro-benchmarks:

- Memory-based tasks: $N \times N$ matrix multiplications, where N is tunable to scale memory pressure.
- Latency-based tasks: a synthetic task that blocks for a configurable sleep duration T to emulate downstream I/O or service latency.

For each task type, we evaluate three benchmark intensities: Light, Medium, and Heavy. For latency-based tasks, each request performs a blocking sleep with $T \in \{100\,\mu s, 1\,ms, 10\,ms\}$ to represent Light, Medium, and Heavy service times, respectively. For memory-based tasks, we use $N \times N$ matrix multiplication with $N \in \{128, 256, 512\}$ to represent Light, Medium, and Heavy working-set sizes, respectively. For fairness, we fixed the CPU resources available to the benchmark by restricting it to exactly 10 cores using tasket.

We evaluated the following experiment matrix (Table 1):

Dimension	Values
Traffic Patterns	Steady load, Burst Load, Ramping Load
Task Types	Memory-based, Latency-based
Task Intensities	Light, Medium, Heavy
Pool Types	Static, Dynamic
Core Counts	10 cores

Table 1: Experiment configuration matrix.

For each task arrival pattern, we measured throughput (tasks completed per second), latency (p50, p95, p99 time from task submission to completion), Memory usage, and thread count (active threads over time). We fixed the CPU resources available to the benchmark by restricting 10 cores. For each configuration, we run the benchmark for 120 seconds and repeat the process three times. we will then aggregate the metrics across the three trials to account for system noise and variability.

4 Results

As referenced in section 3.3, we ran a total of 36 configurations (3 traffic patterns × 2 task types × 3 task intensities × 2 pool types) with three trials each, resulting in 108 benchmark runs. For these experiments, we set the static thread pool to have a fixed size of 10 threads, matching the number of CPU cores that were pinned. For the dynamic thread pool, we set a minimum thread count of 0 and a rate of spawning a new thread for every 10 tasks in the queue, with the maximum set to OS limits. We did not set a max

thread count for the dynamic thread pool as at a cap the dynamic pool would behave like the static pool and we wanted to observe the effects of over-scaling.

4.1 Latency-Based Tasks

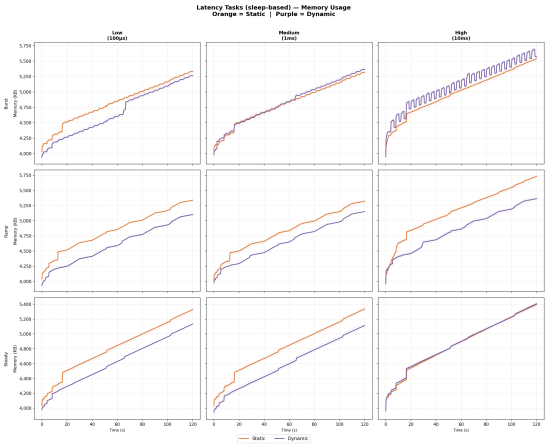
Table 2 reports mean throughput, latency percentiles, memory, and thread count for all latency-based configurations averaged over three trials. Figure 2 shows the corresponding time-series traces. As shown in the table and the figures, for light (100 μs) and medium (1 ms) latency tasks, both pools perform similarly in throughput (999 t/s), latency (p99 around 1.1ms) and queue depth (near zero) and we see no benefit or cost of dynamic scaling. However, for the heavy (10 ms) latency tasks, we see that the static thread pool saturates as 10 threads and 10 ms task time equals the rate of 1000 t/s, which causes zero headroom for queuing and results in the queue depth spiking to thousands of pending tasks and p99 latency shooting up to seconds. In contrast, the dynamic thread pool scales up to aruond 15 threads, which allows it to keep up with the incoming task rate and maintain a p99 latency of around 10 ms with similar memory usage to the static pool.

Under heavy latency tasks, we can see that the static thread pool shows a p99 latency difference of about 7x under different traffic patterns (Burst: 2591 ms, Ramp: 5617 ms, Steady: 865 ms) while the latency remains consistent with respect to task size for the static pool under light and medium tasks, which shows that the traffic pattern only matters when the pool is near saturation. On the other hand, the dynamic thread pool maintains a consistent p99 latency of around 10 ms across all traffic patterns for the heady latency tasks, which shows that the dynamic pool can effectively handle the varying traffic patterns for latency-sensitive workloads.

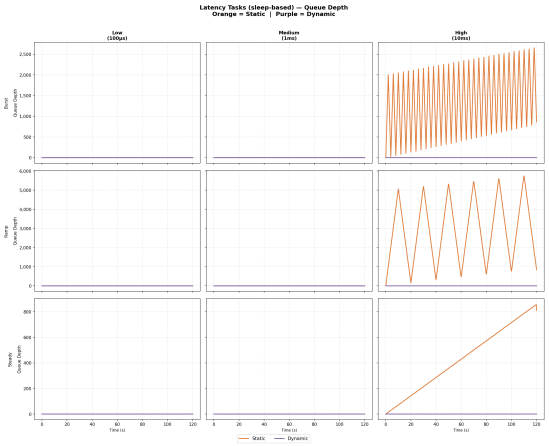
These results show us that for latency-sensitive workloads, dynamic thread pools can provide significant performance benefits as the threads do not have to do much active work and can simply block on the task.

4.2 Matrix Multiplication Tasks

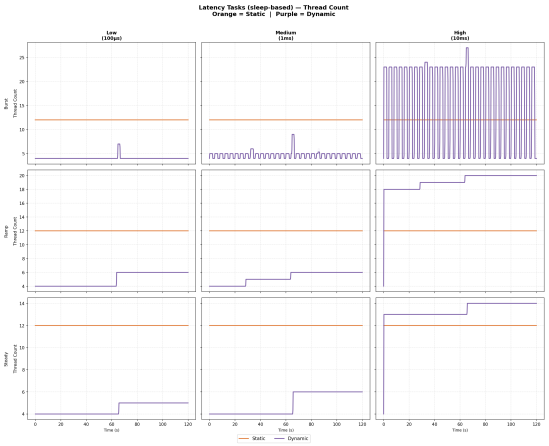
Table 3 reports mean throughput, latency percentiles, memory, and thread count for all matrix multiplication configurations averaged over three trials. Figure 3 shows the corresponding time-series traces. As shown in the table and the figures, for the light (128 x 128) matrix multiplication tasks, both pools perform, similarly in throughput (999 t/s), latency (p99 around 6 ms) and queue depth (near zero). The only exception being under burst traffic where the dynamic pool spawns more threads than the static pool, which causes it to use more memory but performance is similar to the static pool. However, for the medium (256 x 256) and heavy (512 x 512) even though the dynamic thread pool performs better in terms of latency, the memory usage and active thread count is significantly higher than the static pool, as shown in Fig 3a and Fig 3c, while the throughput crashes. This happens because with the creation of more threads, the main overhead of the dynamic thread pool changes to the overhead of context switching and memory bloat from excess threads, as a lot of time and resources are being used to manage the threads instead of doing the actual work of multiplying the matrices.



(a) Memory usage



(b) Queue depth over time



(c) Active thread count over time

Figure 2: System metrics for latency-based tasks across all traffic patterns and intensities.

Traffic	Task	Pool	Tput (t/s)	p99 (ms)	Mem (MB)	Threads
Burst	Light	Static	998.9	0.20	4.7	12.0
		Dynamic	998.9	0.20	4.6	4.0
	Medium	Static	998.9	1.10	4.8	12.0
		Dynamic	998.9	1.10	4.8	4.7
	Heavy	Static	992.3	2591.75	4.9	12.0
		Dynamic	998.9	10.10	5.0	15.5
Ramp	Light	Static	998.9	0.20	4.8	12.0
		Dynamic	998.9	0.20	4.6	4.9
	Medium	Static	998.9	1.11	4.8	12.0
		Dynamic	998.9	1.10	4.6	5.2
	Heavy	Static	992.3	5617.00	5.1	12.0
		Dynamic	998.9	10.11	4.8	19.2
Steady	Light	Static	998.9	0.19	4.8	12.0
		Dynamic	998.9	0.19	4.6	4.5
	Medium	Static	998.9	1.09	4.8	12.0
		Dynamic	998.9	1.10	4.6	4.9
	Heavy	Static	992.3	865.42	4.8	12.0
		Dynamic	998.9	10.10	4.8	13.4

Table 2: Mean results across three trials for latency-based tasks. Mem is mean usage in MB; Threads is mean active thread count.

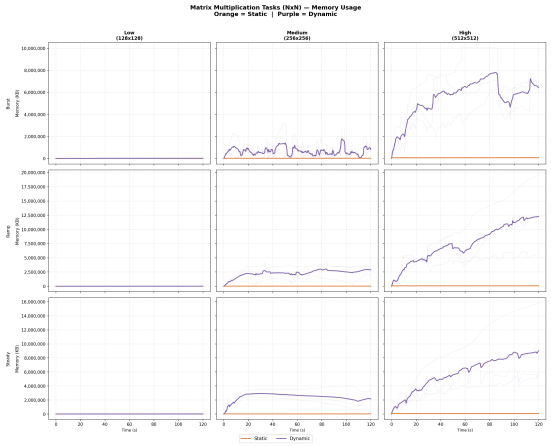
Under different traffic patterns, the performance of the static pool is consistent across all traffic pattern for a specific task size, which shows that the traffic pattern does not matter when the pool is saturated and the bottleneck is the work being done by the threads. The dynamic thread pool also shows a similar performance across all traffic patterns for a specific task size, which shows that the traffic pattern does not matter when the workload is the main bottleneck.

This shows us that for the CPU-bound workloads, the static thread pool can perform better as the threads are doing active work and the overhead of managing the threads in the dynamic pool can outweigh the benefits of scaling and ideally some sort of thread cap should be used to prevent over-scaling and the performance degradation.

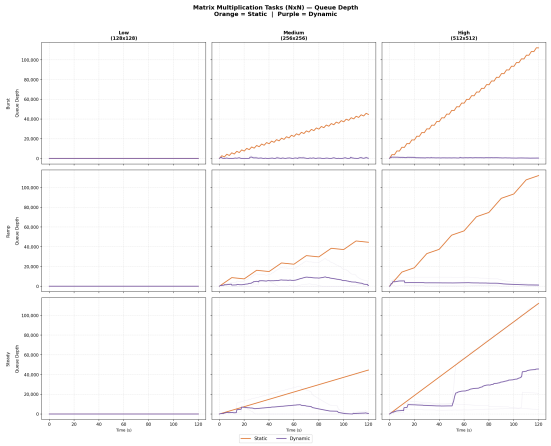
5 Conclusion

In this project, we aim to evaluate the performance trade-offs between static and dynamic thread pool architectures under varying workloads and traffic patterns. We can summarize our findings as follows:

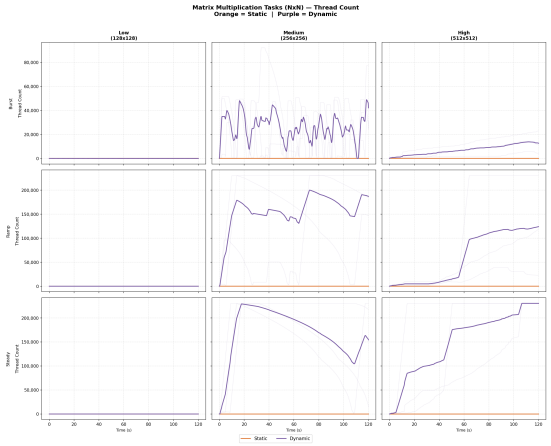
- Dynamic pool wins for I/O-bound / latency-sensitive workloads : thread scaling prevents queue buildup when tasks can block. This advantage increases with the task duration and traffic.
- Static pool wins for CPU-bound workloads: A fixed thread count matching available cores avoids context-switching overhead, cache thrashing, and memory bloat from excess threads. While the static pool can saturate under high load, the dynamic pool's performance degrades more due to context switching overhead.
- Thread over-scaling is harmful under compute pressure: spawning threads beyond core count produces diminishing returns that actively hurts throughput.



(a) Memory usage



(b) Queue depth over time



(c) Active thread count over time

Figure 3: System metrics for matrix multiplication tasks across all traffic patterns and intensities.

Traffic	N	Pool	Tput (t/s)	p99 (ms)	Mem (MB)	Threads
Burst	128	Static	999.0	5.74	7.9	12.0
		Dynamic	998.9	6.62	16.5	17.9
	256	Static	628.5	71463.92	21.1	12.0
		Dynamic	252.0	7267.56	549.8	23420.4
	512	Static	65.8	1687830.86	72.1	12.0
		Dynamic	62.2	48941.92	4820.7	7125.4
Ramp	128	Static	998.9	6.56	8.1	12.0
		Dynamic	998.9	6.59	8.7	14.0
	256	Static	628.5	72535.67	21.6	12.0
		Dynamic	253.3	44085.01	2091.2	137086.4
	512	Static	65.9	1685673.80	72.1	12.0
		Dynamic	60.4	151306.84	7219.7	43557.9
Steady	128	Static	998.9	6.60	8.0	12.0
		Dynamic	998.8	6.55	7.4	10.0
	256	Static	628.0	70277.60	21.4	12.0
		Dynamic	185.1	38719.17	2156.3	150466.3
	512	Static	65.8	1685253.26	72.1	12.0
		Dynamic	55.1	841820.59	7395.1	133053.4

Table 3: Mean results across three trials for matrix multiplication tasks ($N \times N$ matrices). Mem is mean usage in MB; Threads is mean active thread count.

- Traffic pattern matters only when the pool is near saturation : Burst and Ramp show queuing instability in the static pool under heavy latency tasks, but have negligible effect when throughput is the bottleneck (e.g. large matrix multiplication)

Overall, our results suggest that the choice between static and dynamic thread pools should be primarily guided by the workload characteristics and how much active work the threads are expected to do.

Note: We used LLM tools like claude, gemini and copilot to help with debugging our implementation and refactoring and cleaning up our benchmark code.

References

[1] George K Adam. 2022. Co-design of multicore hardware and multithreaded software for thread performance assessment on an FPGA. *Computers* 11, 5 (2022), 76.

[2] Daniela L Freire, Angela Mazzonetto, Rafael Z Frantz, Fabricia Roos-Frantz, Sandro Sawicki, and Vitor Basto-Fernandes. 2021. Performance evaluation of thread pool configurations in the run-time systems of integration platforms. *International Journal of Business Process Integration and Management* 10, 3-4 (2021), 318–329.

[3] DongHyun Kang, Saeyoung Han, SeoHee Yoo, and Sungyong Park. 2008. Prediction-based dynamic thread pool scheme for efficient resource usage. In *2008 IEEE 8th International Conference on Computer and Information Technology Workshops*. IEEE, 159–164.

[4] Ji Hoon Kim, Seungwok Han, Hyun Ko, and Hee Yong Youn. 2007. Prediction-based dynamic thread pool management of agent platform for ubiquitous computing. In *International Conference on Ubiquitous Intelligence and Computing*. Springer, 1098–1107.

[5] Kang-Lyul Lee, Hong Nhat Pham, Hee-seong Kim, Hee Yong Youn, and Ohyoung Song. 2011. A Novel Predictive and Self – Adaptive Dynamic Thread Pool Management. In *2011 IEEE Ninth International Symposium on Parallel and Distributed Processing with Applications*. 93–98. doi:10.1109/ISPA.2011.61

[6] Yibei Ling, Tracy Mullen, and Xiaola Lin. 2000. Analysis of optimal thread pool size. *ACM SIGOPS Operating Systems Review* 34, 2 (2000), 42–55.